

二维数组

复习

- | | |
|---|----|
| 1. 为什么要多文件编程 | 了解 |
| 2. 多文件编程具体步骤 | 重点 |
| 3. <code>include <></code> 和 <code>#include ""</code> 的区别 | 熟悉 |
| 4. 防止头文件重复包含 | 重点 |

目录

- | | |
|---------------|----|
| 1. 二维数组概述 | 了解 |
| 2. 二维数组的声明与定义 | 熟悉 |
| 3. 二维数组的初始化 | 重点 |
| 4. 二维数组的下标 | 重点 |
| 5. 二维数组循环赋值 | 重点 |
| 6. 二维数组循环取值 | 重点 |
| 7. 二维字符数组 | 重点 |
| 8. 二维数组应用 | 重点 |

二维数组概述

先前学习了快速处理多个变量的问题，可以看成是处理一条线上的问题。

那么如果是一个面上（**行列**）的问题呢？

这个时候需要二维数组了！

C++ 的数组是支持多维的，如果是二维数组，那么数组的**每一个元素都是一个一维数组**，如果数组是三维数组，那么每一个元素都是一个二维数组。

一般我们使用最多的就是二维数组，很少用到三维数组，更多维的数组初学时几乎更用不到。

二维数组的声明与定义

定义

C++ 的二维数组，就是数组的每一个元素都是一个一维数组。

语法

```
type arrName[row][col];
```

数据类型 数组名[行数][列数];

二维数组的声明与定义

声明一个3行6列的整型数组

```
int arrnum[3][5];
```

二维数组的初始化

1.可先声明之后在初始化

```
int arrnum[2][2];
```

```
arrnum[0][0] = 100;
```

```
arrnum[0][1] = 200;
```

```
arrnum[1][0] = 300;
```

```
arrnum[1][1] = 400;
```

2.声明的时候即可初始化

```
int arrnum[2][2] = {100,200,300,400};
```

```
int arrnum[2][2] = {{100,200},{300,400}}; //推荐写法
```

```
int arrnum[][2] = {100,200,300,400};
```

二维数组的下标

二维数组可以看成是一个面上的行列关系，所以下标既有行又有列

行		0	1	2
列	0	<code>arr[0][0]</code> 1	<code>arr[0][1]</code> 2	<code>arr[0][2]</code> 3
	1	<code>arr[1][0]</code> 4	<code>arr[1][1]</code> 5	<code>arr[1][2]</code> 6

下标都是从0开始

二维数组循环赋值

二维数组大部分时候都是行列比较多，这个时候如果一个一个赋值，比较繁琐，所以选择循环赋值的方式比较便捷。

二维数组循环赋值

//声明数组

```
int arr[10][5];
```

//循环赋值

```
for(int i=0; i<10; i++)
```

```
    for(int j=0; j<5; j++)
```

```
        arr[i][j] = i*10 + j;
```

二维数组循环取值

//声明数组

```
int arr[3][6] = {1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18};
```

//按行列输出

```
for(int i=0;i<3;i++)
```

```
    for(int j=0;j<6;j++)
```

```
        cout << arr[i][j] << " ";
```

```
    cout << endl;
```

循环赋值取值案例

通过代码赋值，在打印如下图案：

1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5

0 0 0 0 0 0
1 1 1 1 1 1
2 2 2 2 2 2
3 3 3 3 3 3
4 4 4 4 4 4
5 5 5 5 5 5
6 6 6 6 6 6

1 0 0 0 0 1
0 1 0 0 1 0
0 0 1 1 0 0
0 0 1 1 0 0
0 1 0 0 1 0
1 0 0 0 0 1

循环赋值取值案例

通过代码赋值，在打印如下图案：

```
1 2 3 4 5  
1 2 3 4 5  
1 2 3 4 5  
1 2 3 4 5  
1 2 3 4 5
```



循环赋值取值案例

通过代码赋值，在打印如下图案：

```
0 0 0 0 0 0
1 1 1 1 1 1
2 2 2 2 2 2
3 3 3 3 3 3
4 4 4 4 4 4
5 5 5 5 5 5
6 6 6 6 6 6
```



每行不同，行上递增

循环赋值取值案例

通过代码赋值，在打印如下图案：



```
1 0 0 0 0 1
0 1 0 0 1 0
0 0 1 1 0 0
0 0 1 1 0 0
0 1 0 0 1 0
1 0 0 0 0 1
```

二维字符数组

二维字符数组其实跟其他类型数组差不多，但在初始化的时候可以简化；可以看成多个一维字符数组组成了二维字符数组。

```
char arr[3][10] = {"Hello","world","C++" } ;//每个字符串占一行
```

```
char arr[][10] = {"Hello","world","C++"} ;//第二维同样不可省略
```

小练习

哪一个二维数组定义是正确的？

```
char ar1[][10]={"abc","bbc","cba"};
```

```
char ar2[3][]={"ab","bbc","cba"};
```

```
int ar3[2][4]={1,2};
```

```
int ar4[4][2]={{1,0},{1},{0},{0,0}};
```

小练习

哪一个二维数组定义是正确的？

`char ar1[][10]={"abc","bbc","cba"};` ✓

`char ar2[3][]={"ab","bbc","cba"};` ✗

`int ar3[2][4]={1,2};` ✓

`int ar4[4][2]={{1,0},{1},{0},{0,0}};` ✓

二维数组应用1 题目

用1-100的随机数填充一个n行m列的整型数组（n和m均小于100），输出每行每列的和。

输入用例：

3 5

1 2 3 4 5

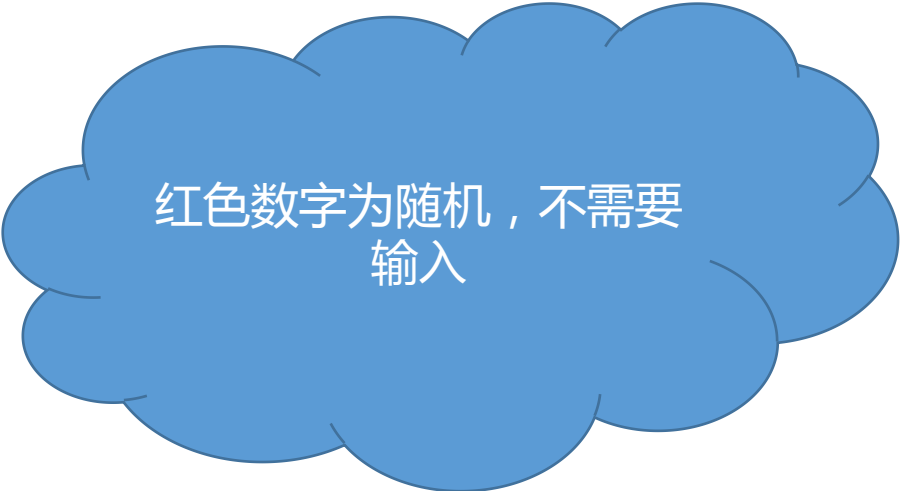
5 2 3 4 1

6 7 8 1 2

输出用例：

15 15 24

12 11 14 9 8



红色数字为随机，不需要
输入

二维数组应用1 代码

```
#include <iostream>
using namespace std;
int arr[100][100];
int row[100];
int col[100];
int main() {
    int n,m,total=0;
    cin >> n >> m;
    //接收数据
    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            cin >> arr[i][j];
        }
    }
    //按行列求和
    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            row[i] += arr[i][j];
            col[j] += arr[i][j];
        }
    }
    //输出行
    for(int i=0; i<n;i++){
        cout << row[i] << " ";
    }
    cout << endl;
    //输出列
    for(int i=0; i<m;i++){
        cout << col[i] << " ";
    }
    return 0;
}
```

二维数组应用2 题目

计算矩阵边缘元素之和

描述输入一个整数矩阵,计算位于矩阵边缘的元素之和。所谓矩阵边缘的元素,就是第一行和最后一行的元素以及第一列和最后一列的元素。输入第一行分别为矩阵的行数 m 和列数 n ($m < 100$, $n < 100$),两者之间以一个空格分开。

接下来输入的 m 行数据中,每行包含 n 个整数,整数之间以一个空格分开。

输出输出对应矩阵的边缘元素和。

样例输入:

3 3

3 4 1

3 7 1

2 0 1

样例输出:

15

二维数组应用2 代码

```
#include <iostream>
using namespace std;
int arr[100][100];
int main() {
    int n,m,total=0;
    cin >> n >> m;
    //接收数据
    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            cin >> arr[i][j];
        }
    }
    //求和
    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            if(i==0 || j==0 || i==n-1 || j==m-1){
                total += arr[i][j];
            }
        }
    }
    //输出
    cout << total << endl;
    return 0;
}
```

二维数组应用3 题目

对字符串中的所有单词进行倒排。

构成单词的字符只有26个大写或小写英文字母，单词与单词间用空格隔开，要求倒排后的单词间隔符以一个空格表示，每个单词最长20个字母。

数据范围：字符串长度不超过100

输入描述：

输入一行，表示用来倒排的句子

输出描述：

输出句子的倒排结果

输入用例：

this is c plus plus

输出用例：

plus plus c is this

二维数组应用3 代码(一维数组)

```
#include <iostream>
#include <cstring>
using namespace std;
char c[100];
int main(){
    gets(c);
    int len = strlen(c);
    int begin=len-1,end=len-1;
    for(int i=len-1;i>=0;i--){
        if(c[i] == ' ' || i == 0){
            end = i;
            int j=0;
            if(i != 0){
                j = end + 1;
            }
            else{
                j = end;
            }
            for(;j<=begin;j++){
                cout << c[j];
            }
            cout << " ";
            begin = end - 1;
        }
    }
    return 0;
}
```

二维数组应用3 代码(二维数组)

```
#include <iostream>
using namespace std;
char c[100][20];
int main(){
    int i=1;
    while(cin >> c[i++]);
    i--;
    while(--i>=1){
        cout << c[i] << " ";
    }

    return 0;
}
```

知识点总结

- | | |
|---------------|----|
| 1. 二维数组概述 | 了解 |
| 2. 二维数组的声明与定义 | 熟悉 |
| 3. 二维数组的初始化 | 重点 |
| 4. 二维数组的下标 | 重点 |
| 5. 二维数组循环赋值 | 重点 |
| 6. 二维数组循环取值 | 重点 |
| 7. 二维字符数组 | 重点 |
| 8. 二维数组应用 | 重点 |

作业

矩阵转置

描述输入一个 n 行 m 列的矩阵 A ,输出它的转置 A^T 。

输入格式:第一行包含两个整数 n 和 m ,表示矩阵 A 的行数和列数。 $1 \leq n \leq 100, 1 \leq m \leq 100$ 。接下来 n 行,每行 m 个整数,表示矩阵 A 的元素。相邻两个整数之间用单个空格隔开,每个元素均在 $1 \sim 1000$ 之间。

输出格式:输出 m 行,每行 n 个整数,为矩阵 A 的转置。相邻两个整数之间用单个空格隔开。

样例输入:

```
3 3
1 2 3
4 5 6
7 8 9
```

样例输出:

```
1 4 7
2 5 8
3 6 9
```

下节课预告

综合案例3